Follow me behavior (detection part)

Olivier Aycard
Professor
Grenoble INP - PHELMA
GIPSA Lab
olivier-aycard.github.io/
olivier.aycard@grenoble-inp.fr



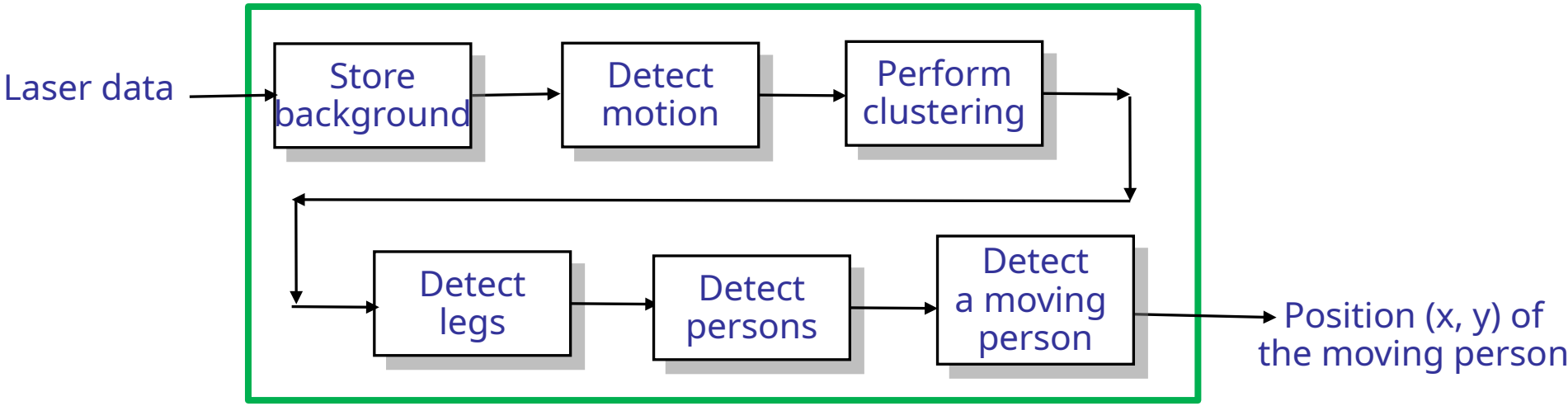


Follow me behavior (perception part): definition (1/2)

- The goal of the lab is to implement the laser processing to detect a moving person;
- Our model of a moving person has:
  - Two legs that are moving;
  - Two legs with a maximum distance of 70cms between them;
  - A leg is a cluster with a size between 5cms and 25cms;
  - A moving cluster is a cluster that has at least 75% of its hits that are dynamic;

Follow me behavior (perception part): definition (2/2)





- Each time, we receive new laser data, this process is done

Follow me behavior (perception part): implementation

1. The node that will detect a moving person is in ~/ros2_ws/src/follow_me/src/nodes/detect_a_moving_person_node.cpp
   Nothing to implement here. This is the main node update loop which calls the methods, that you have to implement in ~/ros2_ws/src/classes
   You have to read and understand the method Update
2. We will start with the method detect_motion() of the class motion_detector
3. The 4 methods, you have to implement, are in ~/ros2_ws/src/follow_me/src/classes/motion_detector.cpp
4. The specification of the class is in ~/ros2_ws/src/follow_me/inc/motion_detector.hpp

Follow me behavior (perception part): implementation

1. We will start with the method detect_motion() of the class motion_detector

```
void moving_person_detector::update()
{
  if (!scan_.get_init_laser()) {
    RCLCPP_WARN_THROTTLE(scan_.get_logger(), *scan_.get_clock(), 2000,
      "Waiting for laser data: launch a rosbag or connect the robot.");
  } else if (!robot_.get_init_robot()) {
    RCLCPP_WARN_THROTTLE(scan_.get_logger(), *scan_.get_clock(), 2000,
      "Waiting for robot state: ros2 run action robot_moving_node");
  } else if (scan_.get_new_laser() && robot_.get_new_robot()) {

    scan_.display_field_of_view();

    motion_.detect_motion();
    motion_.display_motion();

    clusters_.perform_clustering();
    clusters_.display_clustering();

    legs_.detect_legs();
    legs_.display_legs();

    persons_.detect_persons();
    persons_.display_persons();

    detect_a_moving_person();
    display_a_moving_detected_person();

    scan_.update_state_variables();
    robot_.update_state_variables();
  }
}
```

Follow me behavior (perception part): implementation

1. The specification of the class is in ~/ros2_ws/src/follow_me/inc/motion_detector.hpp
2. The 4 methods, you have to implement, are in ~/ros2_ws/src/follow_me/src/classes/motion_detector.cpp

```
class motion_detector
{
protected:
  // Non-owning references: inputs must outlive this object.
  const laser & scan_;
  const robot_state & robot_;

  // Internal state: reference ranges.
  std::vector<float> background_;

private:
  // Output: motion flag for each beam.
  std::vector<bool> dynamic_;

  // Visualization: marker and publisher for displaying detected motion in RViz.
  visualization_msgs::msg::Marker marker_motion_;
  rclcpp::Publisher<visualization_msgs::msg::Marker>::SharedPtr pub_motion_marker_;

public:
  // The node creates publishers; dependencies are kept by reference.
  explicit motion_detector(rclcpp::Node & node, const laser & scan, const robot_state & robot);
  virtual ~motion_detector() = default;

  // Four functions to implement.
  virtual void detect_motion(float threshold = detection_threshold);
  virtual void store_background();
  virtual void reset_motion();
  virtual void detect_current_motion(float threshold = detection_threshold);
```

Follow me behavior (perception part): installation + implementation

1. In ~/ros2_ws/src/follow_me/src/nodes/detect_a_moving_person_node.cpp:
   Nothing to implement here. This is the main node update loop which calls the methods, that you have to implement in ~/ros2_ws/src/classes
   You have to read and understand the method Update.

2. In ~/ros2_ws/src/follow_me/src/datmo.cpp:
   You have to implement all the methods in the file
   1. Store_background ;
   2. Reset_motion ;
   3. Detect_current_motion ;
   4. Detect_motion;

   5. Perform_basic_clustering ;
   6. Perform_advanced_clustering;
   7. Detect_legs;

Follow me behavior (perception part): installation + implementation

1. In ~/ros2_ws/src/follow_me/src/nodes/detect_a_moving_person_node.cpp:
   Nothing to implement here. This is the main node update loop which calls the methods, that you have to implement in ~/ros2_ws/src/classes
   You have to read and understand the method Update.

2. In ~/ros2_ws/src/follow_me/src/datmo.cpp:
   You have to implement all the methods in the file
   1. Store_background ;
   2. Reset_motion ;
   3. Detect_current_motion ;
   4. Detect_motion;

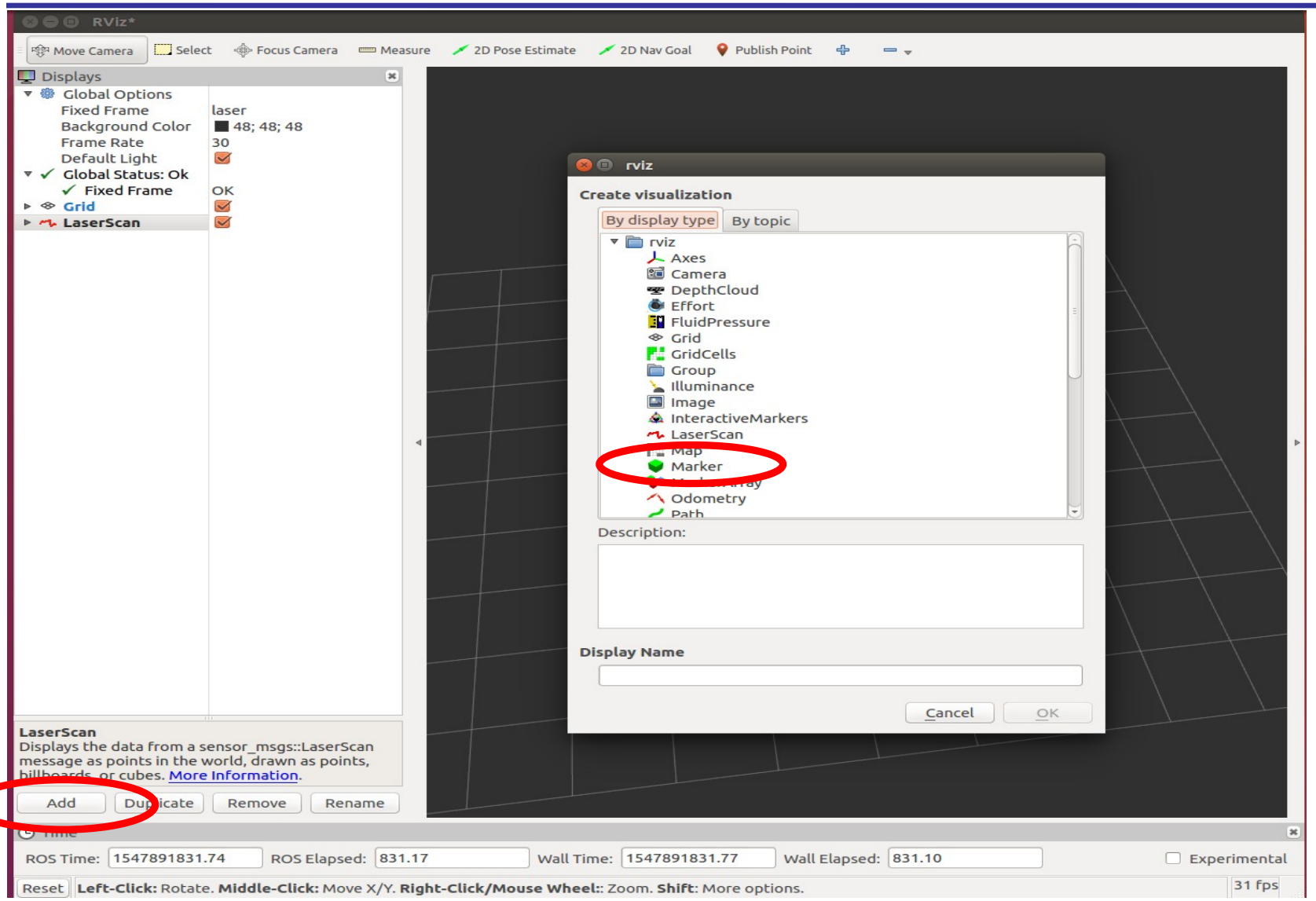
   5. Perform_basic_clustering ;
   6. Perform_advanced_clustering;
   7. Detect_legs;

Follow me behavior (perception part): How to run

- Open 2 terminals:
  1. Terminal to run your nodes and Rviz :
  2. `cd ~/ros2_ws/src/follow_me/scripts`
     `./start_robair_detection_only.sh`
  3. [Optional] Terminal to play a rosbag file if a robot is not available :
     `cd ~/ros2_ws/data_for_labs/follow_me/detection/old_laser`
  4. `ros2 bag play <data_file>.bag2`

- **Alternative option :**
  Instead of one terminal for all ros nodes using the script, you can also create one tab for each node in a terminal, and run the command for each node and Rviz individually.
  Open the scripts under follow_me/src/scripts to find the commands of the form :
  `ros2 run <package_name> <node_name>`

Follow me behavior (perception part): Rviz



Follow me behavior (perception part): Rviz

